



内蒙古大学
INNER MONGOLIA UNIVERSITY

《深度学习》设计报告

学院 : 电子信息工程学院

年级 : 2023 级

专业 : 通信工程（产教融合）

姓名 :

学号 :

日期 : 2026 年 5 月 12 日

摘要与方法

本实验使用课程提供的 MNIST 数据，编写数据读取、分批近邻搜索、PCA、投票与评估程序。先在训练集内部固定划分上比较参数，再用完整训练集和独立测试集评估选定方案。关键实现摘录列在正文，完整程序见文末网页。

方法与选型

kNN 按样本间距离寻找 k 个近邻，再按类别票数预测。L2 搜索计算平方距离 $\|x\|^2 + \|x_i\|^2 - 2x \cdot x_i$ ；平方不改变近邻顺序。PCA 的均值和投影基仅由当前训练部分计算，验证集与测试集只做映射。

在 12,000 张训练、2,000 张验证图像上，PCA-50、L2、均匀投票的 $k=3$ 达到 96.80% Top-1。距离加权仅作为对照；程序中的 d 是平方 L2 距离，加权分数实际为 $1/\max(d, \epsilon)$ ，零距离邻居优先。

最终结果

选定配置后，用全部 60,000 张训练图像重新拟合 PCA，在 10,000 张测试图像上得到 Top-1 97.52%、Top-3 99.05%、Top-5 99.30%。Top-3/5 依代码中的类别得分排序；未获票类别得分为零，因此这些指标需要连同 Top-1 和同分规则解读。

关键词：MNIST； k 近邻；PCA；Top- n ；图像分类

数据、环境与评价设计

课程仓库 KNN.zip 包含 MNIST 的 IDX 数据。训练集 60,000 张、测试集 10,000 张，每张为 28×28 灰度图，标签为 0-9。原始像素除以 255；图像来源于压缩包内同一份数据，避免重复读取。

探索阶段使用随机种子 42，从原训练集每类无放回抽取 1,200 张训练图像，再从剩余样本每类抽取 200 张验证图像。所有候选配置共用这 12,000/2,000 张的划分。测试集不参与选型；最终配置使用完整训练集重新拟合。

运行环境与指标

Windows; NumPy、scikit-learn、scikit-image、Matplotlib; 完整评估用 CUDA PyTorch 在 NVIDIA GeForce RTX 4080 Laptop GPU 上做分批距离计算。探索阶段用 CPU。原始像素统一除以 255；PCA 为自行实现的训练集协方差特征分解。

Top-1 为预测标签正确率；Top-3/5 为真实标签出现在按投票分数排序的前 3/5 类中的比例。混淆矩阵行是真实类别，列是预测类别；逐类准确率为该行对角元素除以该行总数。计时分别记录特征变换、数据保存式 fit、近邻搜索；k 扫描复用同一批 9 个近邻。

课程源码：IMUEEMM/DL-CLASS, pj1-KNN, 提交 feec5b1b1bb93f7b70b369c74fd98a1bf7eaab91。

分批 kNN：距离、候选与排序

课程脚本提供数据及 sklearn 基线。本实验另写 experiment.py, 采用标准的 kNN 数学方法。下列摘录来自实际运行的 CPU 搜索函数 neighbors_numpy(), 各行按执行顺序排列。

CPU 代码摘录 (experiment.py)	作用
for start in range(0, len(b), batch):	每次只处理一批查询图像
q = b[start:start+batch]	取出当前批次
d = np.maximum((q*q).sum(axis=1, keepdims=True) + norms[None, :] - 2*q@a.T, 0)	计算平方 L2 距离
part = np.argpartition(d, max_k-1, axis=1)[:max_k]	筛出前 9 个候选
order = np.argsort(np.take_along_axis(d, part, axis=1), axis=1, kind="stable")	候选内部按距离排序
idx = np.take_along_axis(part, order, axis=1)	恢复近邻的训练集索引

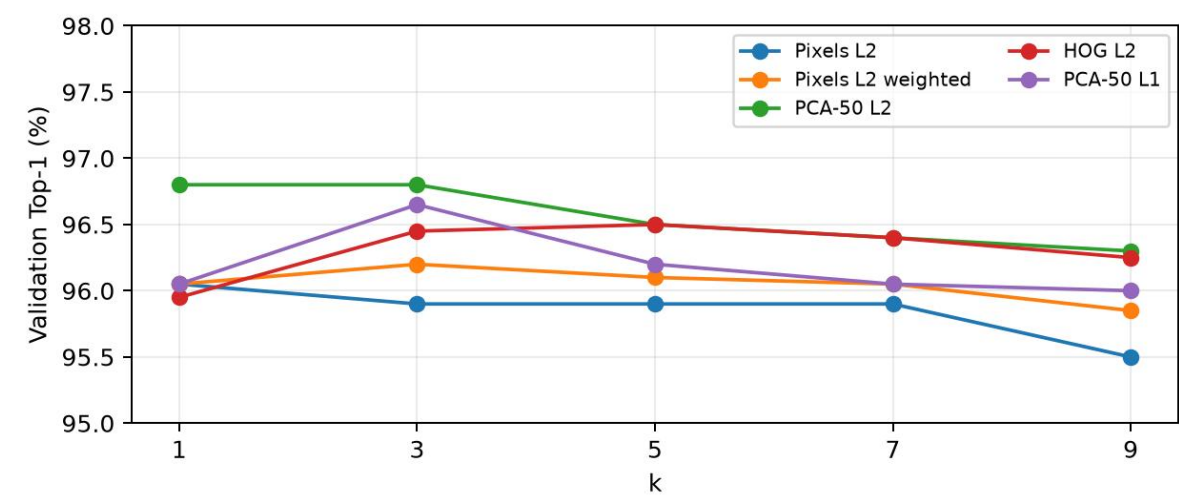
矩阵乘法一次得到当前批次与全部训练图像的距离。CPU 默认每批 64 张；完整测试走 neighbors_torch(), 每批 128 张, 使用相同平方 L2 公式及 torch.topk() 取近邻。这样无需保存整个 10,000×60,000 距离矩阵。

与现成实现核对

同一 12,000/2,000 划分上, sklearn 的欧氏距离暴力搜索与本实现的 k=5 Top-1 均为 95.90%, 2,000 条预测逐条相同。后续比较使用本实验的搜索与投票函数。

不同 k 对分类效果的影响

同一验证划分比较 k=1、3、5、7、9。图中五条曲线分别是原始像素均匀投票、原始像素加权投票、PCA-50 L2、HOG L2、PCA-50 L1 的 Top-1；每种配置都复用该特征的一次近邻搜索结果。



原始像素的结果 (Top-1)

k	1	3	5	7	9
Top-1	96.05%	95.90%	95.90%	95.90%	95.50%

原始像素从 k=1 增到 9 未带来持续改善。PCA-50 在 k=1、3 都达到 96.80% Top-1；k=3 的 Top-3 为 98.65%,高于 k=1 的 96.95%。因此在查看测试结果前,选定 PCA-50、L2、k=3。

PCA 的实现与特征比较

比较 784 维归一化像素、自写 PCA 的 50 维投影，以及 324 维 HOG。HOG 用 9 个方向、7×7 像素单元和 2×2 单元块提取。下表均为同一验证集的 Top-1 (%)。

验证集 Top-1 (%)

特征 / k	1	3	5	7	9
像素	96.05	95.90	95.90	95.90	95.50
PCA-50	96.80	96.80	96.50	96.40	96.30
HOG	95.95	96.45	96.50	96.40	96.25

PCA 关键代码：训练部分拟合，其他部分只投影

experiment.py 中 pca_fit_transform() 摘录	作用
<code>mean = a.mean(axis=0)</code>	训练样本的均值
<code>centered = a - mean</code>	仅中心化训练样本
<code>cov = (centered.T @ centered) / (len(a) - 1)</code>	训练样本协方差
<code>eigenvalues, vectors = np.linalg.eigh(cov)</code>	对称矩阵特征分解
<code>basis = vectors[:, order]</code>	取最大 50 个特征值对应方向
<code>(b-mean) @ basis</code>	验证或测试样本共用训练参数

探索阶段 PCA-50 解释训练样本方差的 82.52%；完整训练集重新拟合后为 82.46%。它在本次验证中多个 k 上优于原始像素，且把搜索维度从 784 降为 50。HOG 在 k=5 为 96.50%，但特征提取另有开销。

投票规则与 Top-n 计算

原始像素 L2 的 k=3 均匀投票与加权投票分别为 95.90%、96.20% Top-1; PCA-50 上, k=3 的 L1 与 L2 分别为 96.65%、96.80%。选型依据是固定验证集的组合结果, 而非只看某一项的最高数值。

验证集 Top-n: PCA-50、L2、均匀投票

k	Top-1	Top-3	Top-5
1	96.80%	96.95%	97.80%
3	96.80%	98.65%	98.95%
5	96.50%	98.85%	99.20%

代码中的类别得分与排名

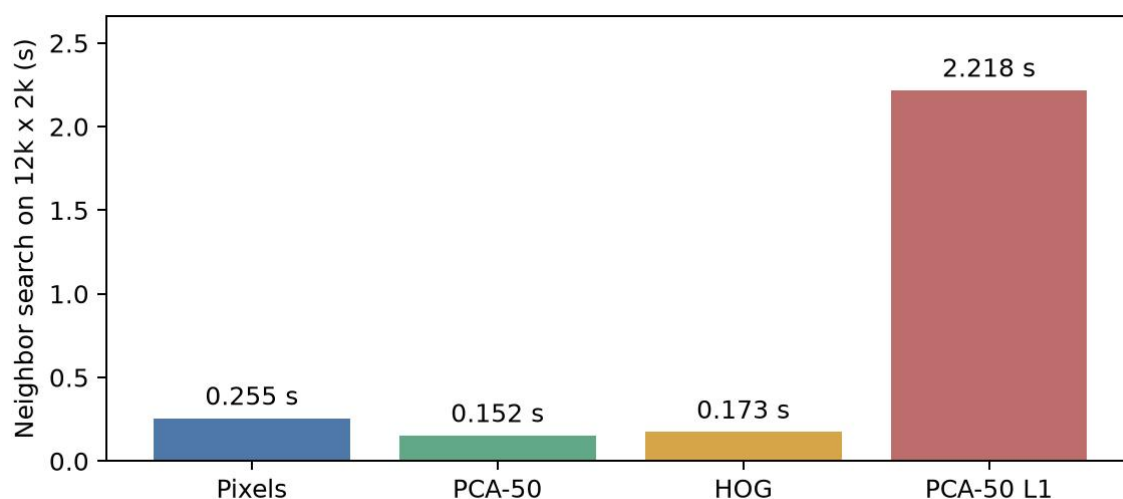
experiment.py 中 vote()/evaluate() 摘录	解释
<code>weights = np.ones_like(dis, dtype=float)</code>	选定方案每个邻居一票
<code>scores[:, c] = ((lab == c)*weights).sum(axis=1)</code>	按数字累计票数
<code>ranking = np.argsort(-scores, axis=1, kind="stable")</code>	得分降序; 同分时数字小者在前
<code>pred, rank = vote(labels, distances, k, weighted)</code>	Top-1 取排名首位
<code>(rank[:, :3] == ytrue[:, None]).any(axis=1)</code>	检查真实类别是否在前 3 位

同分规则也作用于零票类别, 所以 k=1 时 Top-3/5 尤其受排名约定影响。最终测试集的 k=3 指标为 97.52%、99.05%、99.30%, 对应 Top-1/3/5。

运行时间与分批计算

下图比较相同 12,000 张训练、2,000 张验证图像上的 CPU 近邻搜索时间。PCA-50 L1

使用分批广播计算绝对差，L2 使用矩阵乘法；图中时间不包含特征变换。



特征准备方面，像素归一化用 0.015 秒，PCA 拟合及变换用 0.202 秒，HOG 提取用 1.684 秒。原始像素近邻搜索用 0.255 秒；sklearn 基线的 predict 用 2.586 秒。两个实现的库和调用范围不同，这些数值只描述本机本次运行。

完整 60,000/10,000 评估中，PCA 变换用 0.316 秒，CUDA 分批近邻搜索用 1.500 秒。

measure() 中的 fit_s 仅计数据复制与保存，不含搜索；因此不能把该数值解释为模型训练耗时。GPU 搜索计时包含设备传输、结果回传与同步，且与上面的 CPU 小样本规模不同。

每类 10 张真实测试样本

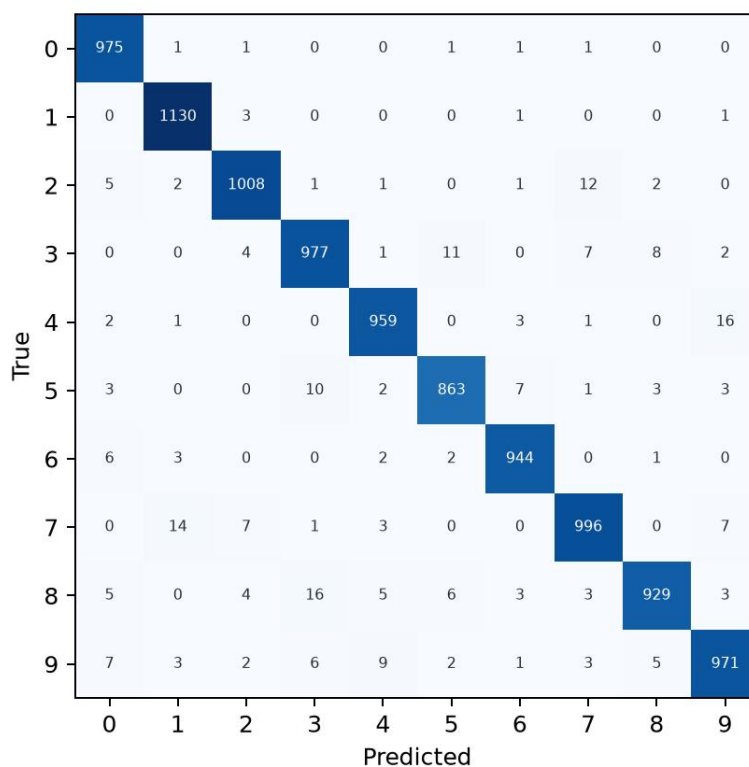
从固定种子 42 的测试集中, 每类无放回选取 10 张, 共 100 张。图中按真实数字 0-9 排成十行; 每张上方为“真实/预测”, 红色表示错误。该展示样本仅用于定性分析, 正式指标来自全部 10,000 张测试图像。



Each row: one true class (0-9); title = true/predicted; red = error

展示样本逐类正确数: 0:10, 1:10, 2:10, 3:10, 4:9, 5:8, 6:9, 7:10, 8:10, 9:10; 合计 96/100。

混淆矩阵、结论与来源



完整测试集正确 9,752/10,000 张。逐类准确率最低的是数字 8 (95.38%)，其次是 9 (96.23%); 8→3 与 4→9 各有 16 例。错误集中在若干笔画相近的类别。

本作业实现并比较标准 kNN、PCA、HOG 方法；没有提出新算法。PCA-50、L2、k=3 是本次固定验证划分选出的配置，97.52% 是该配置在独立测试集上的 Top-1。

数据与教师脚本：

<https://github.com/IMUEEMM/DL-CLASS/tree/feec5b1b1bb93f7b70b369c74fd98a1bf7eaab91/pj1-KNN>

算法参考：scikit-learn KNeighborsClassifier 文档；MNIST 原始数据来源见课程仓库。

报告、图表与完整代码可直接在浏览器中查看：

报告网页：<https://lzcxfvs-maker.github.io/LZCXVFS/>

完整代码：<https://lzcxfvs-maker.github.io/LZCXVFS/#code>